

Alur End-to-End: Issue Out to Production

ERP Geum Cheon (GCI) · APK Material Tracker → Laravel API → Dashboard Web
Dokumen teknis · dibuat dari implementasi aktual pada repository

Tujuan dokumen: menjelaskan perjalanan satu transaksi pengeluaran material (issue out) mulai dari operator memindai label di APK, diproses backend secara atomik dan idempoten, sampai muncul real-time di halaman monitoring web.

1. Gambaran Alur

GARIS BESAR



2. Sisi APK (Flutter)

Langkah	File	Endpoint
Login	repositories/auth_repository.dart	POST /api/auth/login
Simpan token	core/storage/token_store.dart	flutter_secure_storage
Daftar WO	repositories/issue_repository.dart	GET /api/work-orders?status=planned
Konteks release	issue_repository.dart	GET /api/work-orders/{id}/release-context
Resolve tag	issue_repository.dart	POST /api/stock-tags/resolve
Validasi + scan	providers/issue_provider.dart	lokal (lihat di bawah)
Submit release	issue_repository.dart	POST /api/work-orders/{id}/release

2.1 Validasi lokal sebelum kirim

Di `IssueNotifier.addScan()`, tag ditolak lebih awal agar tidak membebani server:

- tag sudah pernah di-scan → `alreadyScanned`
- part tidak dibutuhkan WO ini → `tagNotNeeded`
- satuan tag $\neq$ satuan item (`uom_rm`) → `tagWrongUom`
- sisa kebutuhan 0 atau qty > sisa → `overRequired`

2.2 Idempotency di sisi klien

`_idempotencyKey = Uuid().v4()` dibuat per sesi WO dan di-reset saat `load()` dan `reset()`. Retry jaringan dengan key yang sama **tidak** menghasilkan dokumen issue baru.

3. Sisi Backend (Laravel)

3.1 Otorisasi

`Gate::authorize('issue', $workOrder)` memakai `WorkOrderPolicy::issue`: lolos bila user memiliki `stock.issue` *atau* `work_order.update` *atau* `production.update`.

3.2 Idempotency

`WoService::releaseWithScans()` memeriksa `idempotency_key`. Bila sudah ada, fungsi langsung mengembalikan issue lama tanpa membuat booking baru.

3.3 Transaksi atomik

Seluruh proses dibungkus `DB::transaction`:

Tabel	Isi
material_issues	issue_no, issue_date, issued_by, received_by, status=posted, idempotency_key
work_order_material_bookings	status=booked — hanya booking , stok fisik belum berkurang
material_issue_items	tag, invoice, supplier, qty, uom, price (per tag yang di-scan)
work_orders	status → in_progress, released_at diisi
production_plan_items	WO masuk papan Production Plan

Kekurangan material dicatat sebagai `shortages` dan menurunkan output secara proporsional.

3.4 Semantik stok (penting)

Release = booking, bukan pengurangan stok.

`part_stocks.qty` *tidak* berubah saat release. Yang berubah hanya ketersediaan:

`tersedia = part_stocks.qty - Σ booking(status = booked)`

Stok baru berkurang saat **Production Result** (dicatat di `work_order_consumptions`). Halaman Stock menampilkan kolom *Di-book* dan *Tersedia*.

3.5 Event setelah commit

Setelah transaksi sukses, `event(new MaterialIssuePosted(...))` dijalankan. Event mengimplementasikan `ShouldDispatchAfterCommit` dan `ShouldBroadcast` dengan `connection = 'database'`, sehingga `BroadcastEvent` masuk ke tabel `jobs` dan diproses queue worker.

4. Sisi Web (Monitoring)

4.1 Halaman monitoring

`/material-issues` (`MaterialIssueController::index`) dengan permission `stock.issue`:

- default **hari ini WIB** (Asia/Jakarta), filter tanggal / operator / status / pencarian
- summary: jumlah issue, WO unik, jumlah tag, qty **per UOM** (tidak dijumlah lintas satuan)
- tabel + aksi **detail** dan **cetak bon** (read-only, tanpa aksi ubah/hapus)

4.2 Update real-time

Di `Outgoing/MaterialIssue/Index.vue`:

- `useEcho('issue-out-monitoring', '.material-issue.posted', handler)`
- event masuk → baris disisipkan di atas (hanya bila cocok filter aktif), summary diperbarui, baris di-highlight, toast muncul
- deduplikasi berdasarkan `issue_id`; tanpa polling
- status koneksi live ditampilkan (connected / connecting / reconnecting / disconnected / failed)

4.3 Detail, cetak, dan kanal privat

- `/material-issues/{id}` → `show`

- `/material-issues/{id}/print` → `resources/views/material-issues/print.blade.php`
- Kanal privat `issue-out-monitoring` (`routes/channels.php`) hanya untuk user dengan `stock.issue`

5. Prasyarat agar Real-time Berjalan

1. `php artisan queue:work` — **wajib**; tanpa ini event menumpuk di tabel `jobs`
2. `php artisan reverb:start`
3. Nginx `location /app` (tanpa trailing slash) + header `Upgrade / Connection`
4. `BROADCAST_CONNECTION=reverb`

Desain penting: release dari APK *tidak* bergantung pada Reverb. Bila queue worker atau Reverb mati, transaksi APK tetap sukses (HTTP 200); hanya dashboard web yang tidak ter-update sampai worker/Reverb hidup kembali.

6. Batasan dan Perilaku yang Perlu Diingat

- **Release = booking**, bukan pengurangan stok. Stok berkurang saat Production Result.
- **Idempotency wajib**. APK yang tidak mengirim `idempotency_key` ditolak `422`.
- **Resolve tag butuh** `stock.issue`. Tanpa permission tersebut, akses ditolak `403`.
- **Payload event dibatasi** pada data aman (`issue_no`, `WO`, `operator`, jumlah item/tag, qty per UOM) — tidak memuat harga, invoice, atau supplier.
- Papan Production Plan difilter per tanggal; WO aktif di tanggal lain ditampilkan di panel terpisah.

7. Cara Verifikasi di Lokal

```
php artisan serve
php artisan queue:work
php artisan reverb:start
npm run dev
```

Lalu lakukan release WO dari APK (atau ekuivalen curl), dan buka `/material-issues` memakai user dengan permission `stock.issue`.